

CODING PROBLEMS

Practice Workbook

30 Structured Problems • Easy · Medium · Hard

Easy

6 Problems

Medium

12 Problems

Hard

12 Problems

Easy Level Problems

1. Sum of Two Numbers [Easy]

Write a program that accepts two integers as input and computes their arithmetic sum. This is a foundational exercise to understand basic input/output operations and arithmetic operators in any programming language.

Sample Input	Sample Output
5 3	8

Constraints

- $-10^9 \leq A, B \leq 10^9$ (integers can be negative)
- Input consists of exactly two integers on a single line
- Output must be a single integer on one line

2. Largest of Three Numbers [Easy]

Given three integers, determine and print the largest value among them. This problem reinforces the use of conditional statements (if-elif-else) and comparison operators to build decision-making logic.

Sample Input	Sample Output
12 47 8	47

Constraints

- $-10^6 \leq A, B, C \leq 10^6$
- All three numbers are distinct integers
- Print exactly one integer — the maximum value

3. Reverse a Number [Easy]

Given an integer, reverse its digits and print the result. For example, 1234 becomes 4321. This problem introduces loop-based digit extraction using modulo and integer division, and also tests handling of trailing zeros.

Sample Input	Sample Output
12345	54321

Constraints

- $1 \leq N \leq 10^9$
- Leading zeros in the reversed output should be omitted (e.g., $100 \rightarrow 1$)
- N is a non-negative integer

4. Prime Number Check [Easy]

Determine whether a given integer is a prime number. A prime number is a natural number greater than 1 that has no positive divisors other than 1 and itself. This problem introduces optimized trial division up to the square root of N.

Sample Input	Sample Output
29	Prime

Constraints

- $2 \leq N \leq 10^6$
- Print "Prime" if N is prime, otherwise print "Not Prime"
- Optimize using trial division up to $\sqrt{N}$

5. Palindrome String [Easy]

Given a string, check whether it reads the same forwards and backwards (i.e., it is a palindrome). This problem strengthens string indexing, slicing, and comparison skills. Ignore case sensitivity in your check.

Sample Input	Sample Output
racecar	Palindrome

Constraints

- $1 \leq \text{len}(S) \leq 1000$
- String may contain alphabets and digits only (no spaces)
- Comparison is case-insensitive

6. Array Maximum [Easy]

Given a list of integers, find and return the maximum element without using any built-in max() function. This exercise reinforces iteration through arrays and tracking a running maximum using a variable.

Sample Input	Sample Output
5 3 7 1 9 4	9

Constraints

- $1 \leq N \leq 10^5$
- $-10^9 \leq \text{each element} \leq 10^9$
- Do not use built-in max() or sort() functions

Medium Level Problems

7. Fibonacci Series [Medium]

Print the first N numbers in the Fibonacci sequence, where each number is the sum of the two preceding ones, starting from 0 and 1. This problem reinforces iterative sequence generation and understanding of recurrence relations.

Sample Input	Sample Output
7	0 1 1 2 3 5 8

Constraints

- $1 \leq N \leq 50$
- Output all N numbers on a single line, space-separated
- Series starts with 0, 1

8. Armstrong Number [Medium]

An Armstrong number (also called a narcissistic number) is one where the sum of its digits each raised to the power of the number of digits equals the number itself. For example, $153 = 1^3 + 5^3 + 3^3$. Determine if the input is an Armstrong number.

Sample Input	Sample Output
153	Armstrong

Constraints

- $1 \leq N \leq 10^6$
- Print "Armstrong" or "Not Armstrong"
- Works for 1 to 7-digit numbers

9. Count Vowels and Consonants [Medium]

Given a string of alphabetic characters, count and display the number of vowels (a, e, i, o, u) and consonants separately. This problem reinforces string traversal and character classification logic.

Sample Input	Sample Output
Hello World	Vowels: 3 Consonants: 7

Constraints

- $1 \leq \text{len}(S) \leq 10^4$
- Input may contain uppercase and lowercase letters and spaces
- Spaces are neither vowels nor consonants — skip them

10. Bubble Sort Implementation [Medium]

Implement the Bubble Sort algorithm to sort a list of integers in ascending order without using any built-in sorting functions. Bubble Sort repeatedly compares adjacent elements and swaps them if they are in the wrong order, with each pass bubbling the largest unsorted element to its correct position.

Sample Input	Sample Output
5 64 34 25 12 22	12 22 25 34 64

Constraints

- $1 \leq N \leq 10^3$
- Do not use built-in `sort()` or `sorted()`
- Time complexity expected: $O(N^2)$

11. Binary Search [Medium]

Implement the Binary Search algorithm on a sorted list of integers to find the index of a given target element. Binary Search works by repeatedly dividing the search range in half, achieving $O(\log N)$ time complexity — significantly faster than linear search.

Sample Input	Sample Output
6 2 5 8 12 16 23 12	Found at index 3

Constraints

- $1 \leq N \leq 10^5$
- The list is sorted in ascending order
- Return index (0-based); print "Not Found" if element is absent

12. Matrix Addition [Medium]

Given two matrices of the same dimensions, compute their element-wise sum. This problem reinforces the use of nested loops for 2D data traversal and matrix representation using lists of lists.

Sample Input	Sample Output
1 2 3 / 4 5 6 + 7 8 9 / 1 2 3	8 10 12 5 7 9

Constraints

- $1 \leq \text{rows}, \text{cols} \leq 100$
- Both matrices have the same dimensions
- $-10^4 \leq \text{each element} \leq 10^4$

13. Matrix Multiplication [Medium]

Given two compatible matrices A (of size $M \times N$) and B (of size $N \times P$), compute their product resulting in a matrix of size $M \times P$. This problem deepens understanding of nested loops and the mathematical definition of matrix multiplication: $C[i][j] = \sum A[i][k] * B[k][j]$.

Sample Input	Sample Output
A: $2 \times 2 =$ 1 2 / 3 4 B: $2 \times 2 =$ 5 6 / 7 8	19 22 43 50

Constraints

- $1 \leq M, N, P \leq 50$

- Number of columns in A must equal number of rows in B
- $-10^3 \leq \text{each element} \leq 10^3$

14. Recursive Factorial [Medium]

Compute the factorial of a non-negative integer N using recursion. The factorial of N (written N!) is defined as $N \times (N-1) \times \dots \times 1$, with $0! = 1$ as the base case. This problem demonstrates recursive function design and the call stack.

Sample Input	Sample Output
6	720

Constraints

- $0 \leq N \leq 15$
- Must use a recursive function (no loops)
- Output a single integer

15. Recursive Sum of Digits [Medium]

Given a positive integer, compute the sum of all its individual digits using recursion. For example, $1234 \rightarrow 1+2+3+4 = 10$. The recursion extracts the last digit using modulo (%) and passes the remaining number divided by 10.

Sample Input	Sample Output
9875	29

Constraints

- $1 \leq N \leq 10^9$
- Must use recursion — no loops
- Output is a single non-negative integer

16. Frequency Counter [Medium]

Given a list of integers, count how many times each element appears using a dictionary. Then display each element along with its frequency. This problem teaches hash-map-based counting, a fundamental pattern in competitive programming and data analysis.

Sample Input	Sample Output
1 2 3 2 1 4 2 3	1 → 2 2 → 3 3 → 2 4 → 1

Constraints

- $1 \leq N \leq 10^5$
- Elements are integers in range $1-10^4$
- Output in order of first occurrence

17. Anagram Checker [Medium]

Given two strings, determine whether they are anagrams of each other. Two strings are anagrams if one can be rearranged to form the other (e.g., "listen" and "silent"). The check must be case-insensitive and should ignore spaces.

Sample Input	Sample Output
listen silent	Anagram

Constraints

- $1 \leq \text{len}(S) \leq 10^4$
- Comparison is case-insensitive; ignore spaces
- Print "Anagram" or "Not Anagram"

18. GCD and LCM [Medium]

Given two positive integers, find their Greatest Common Divisor (GCD) and Least Common Multiple (LCM). Use the Euclidean algorithm for efficient GCD computation. LCM is derived as $\text{LCM}(A, B) = (A \times B) / \text{GCD}(A, B)$.

Sample Input	Sample Output
12 18	GCD: 6 LCM: 36

Constraints

- $1 \leq A, B \leq 10^9$
- Use Euclidean algorithm for GCD
- Output GCD and LCM on separate lines

Hard Level Problems

19. Pattern — Diamond [Hard]

Print a full diamond pattern made of asterisks (*) for a given odd size N. The pattern has an upper triangle that expands from 1 to N stars (with appropriate spaces) and a lower triangle that contracts back. This problem requires precise control over spacing logic and nested loops.

Sample Input	Sample Output
5	* * * * * * * * * * * * * * * *

Constraints

- $1 \leq N \leq 99$, and N is odd
- Use spaces for alignment on both sides

- No trailing spaces after the last star on each row

20. Prime Numbers in Range [Hard]

Print all prime numbers between two given integers L and R (inclusive). Use the Sieve of Eratosthenes for efficient computation. The sieve pre-marks composite numbers, delivering primes in $O(N \log \log N)$ time — far faster than checking each number individually.

Sample Input

10 50

Sample Output

11 13 17 19 23 29 31 37 41 43 47

Constraints

- $2 \leq L \leq R \leq 10^6$
- Use Sieve of Eratosthenes for efficiency
- Output all primes space-separated on one line

21. Longest Word in Sentence [Hard]

Given a sentence, find and print the longest word. If multiple words share the same maximum length, print the first one encountered. This problem involves string splitting, iteration, and length comparison — a common text-processing task.

Sample Input

The quick brown fox jumps over the
lazy dog

Sample Output

jumps

Constraints

- $1 \leq \text{len}(\text{sentence}) \leq 10^4$
- Words are separated by single spaces
- If tie, return the first longest word

22. Remove Duplicate Elements [Hard]

Remove all duplicate elements from a list while preserving the original order of first occurrences. You must accomplish this without using Python's built-in `set()` function. This exercise strengthens manual tracking of seen elements using a list or dictionary.

Sample Input

4 2 7 2 4 9 1 7

Sample Output

4 2 7 9 1

Constraints

- $1 \leq N \leq 10^5$
 - Do not use `set()` or `frozenset()`
 - Preserve original order of first occurrence
-

23. File Word Counter [Hard]

Read a text file from disk and compute the total number of words, lines, and characters in it. This is a practical file I/O problem that teaches working with the file system, reading content line by line, and applying string operations to real data.

Sample Input sample.txt (contains 3 lines of text)	Sample Output Lines: 3 Words: 18 Characters: 102
--	--

Constraints

- File must exist in the working directory
- A word is any sequence of characters separated by whitespace
- Characters include all characters including spaces and newlines

24. Class-Based Student Management [Hard]

Design a Student class with instance attributes for name and a list of subject marks. Implement methods to add a mark for a subject, display all marks, and calculate the student's average. This introduces Object-Oriented Programming (OOP) with encapsulation and method design.

Sample Input Name: Alice Marks: 85 90 78 92	Sample Output Student: Alice Marks: [85, 90, 78, 92] Average: 86.25
--	---

Constraints

- Class must include: `__init__`, `add_mark`, `display`, `calculate_average`
- $0 \leq \text{each mark} \leq 100$
- Average rounded to 2 decimal places

25. Banking System Using OOP [Hard]

Build a BankAccount class that simulates a basic banking system. The class should support deposit, withdraw, and check balance operations. This problem reinforces OOP concepts such as encapsulation of state and methods, and adds validation logic (e.g., insufficient funds).

Sample Input Deposit: 5000 Withdraw: 1500 Check Balance	Sample Output Deposited: 5000 Withdrawn: 1500 Balance: 3500
---	---

Constraints

- Initial balance is 0
 - Withdraw must check for sufficient balance
 - All amounts are positive integers
-

26. Employee Inheritance System [Hard]

Model an employee hierarchy using inheritance. Create a base Employee class with name and base salary, then derive Developer (with a programming language attribute) and Manager (with a team size attribute). Override a display() method in each subclass to show role-specific details. This reinforces polymorphism and inheritance in OOP.

Sample Input	Sample Output
Developer: Alice, Python, Salary: 80000 Manager: Bob, Team: 5, Salary: 95000	Alice Developer Python \$80,000 Bob Manager Team: 5 \$95,000

Constraints

- Must use class inheritance (not composition)
- Each subclass must override the display() method
- Salary is a positive integer

27. Multithreaded Number Printer [Hard]

Create two threads that run concurrently: one thread prints all even numbers from 1 to N, and another prints all odd numbers from 1 to N. Use thread synchronization (locks/semaphores) to ensure numbers are printed in the correct interleaved order: 1, 2, 3, 4... This problem introduces concurrent programming and synchronization primitives.

Sample Input	Sample Output
N = 10	1 2 3 4 5 6 7 8 9 10

Constraints

- $2 \leq N \leq 1000$, and N is even
- Use threading module with Lock or Semaphore
- Output must be in strictly ascending order

28. Producer–Consumer Simulation [Hard]

Simulate the classic Producer–Consumer concurrency problem using threads and a shared buffer (queue). The producer thread generates items and adds them to the buffer; the consumer thread retrieves and processes them. Use thread synchronization to prevent race conditions and buffer overflow/underflow. This is a cornerstone problem in concurrent systems design.

Sample Input	Sample Output
Buffer Size: 5 Items to Produce: 10	Produced: 1..10 Consumed: 1..10 All items processed.

Constraints

- Use threading.Semaphore or queue.Queue for synchronization
- Buffer must not overflow (bounded buffer)
- Consumer must wait when buffer is empty

29. Optimize Nested Loop Problem [Hard]

Given a large list of integers, find all duplicate elements using an optimized approach. A naive $O(N^2)$ nested loop solution is unacceptable for large inputs. Instead, use a hash set for $O(N)$ time complexity — tracking elements seen so far and identifying duplicates in a single pass through the list.

Sample Input 10 1 3 4 2 2 3 5 6 4 8	Sample Output Duplicates: 2 3 4
--	---

Constraints

- $1 \leq N \leq 10^6$
- Must achieve $O(N)$ time complexity
- Output duplicates in the order they are first repeated

30. Mini Library Management System [Hard]

Design a fully functional Library Management System using OOP. The system should support: adding new books (with title, author, ISBN), issuing books to users (marking them as unavailable), returning books (marking them available again), and searching books by title or author. This problem integrates multiple OOP concepts — class design, encapsulation, and object relationships — into a real-world mini application.

Sample Input Add: 'Python 101', Author: 'John', ISBN: 1001 Issue: ISBN 1001 to 'Alice' Search: 'Python'	Sample Output Book Added. Issued to Alice. Found: Python 101 by John
--	--

Constraints

- Use separate Book and Library classes
 - Issue must check if book is available
 - Search should be case-insensitive (partial match allowed)
-